

Concurrent and Distributed Systems

Lab 9 – Web services

Materials

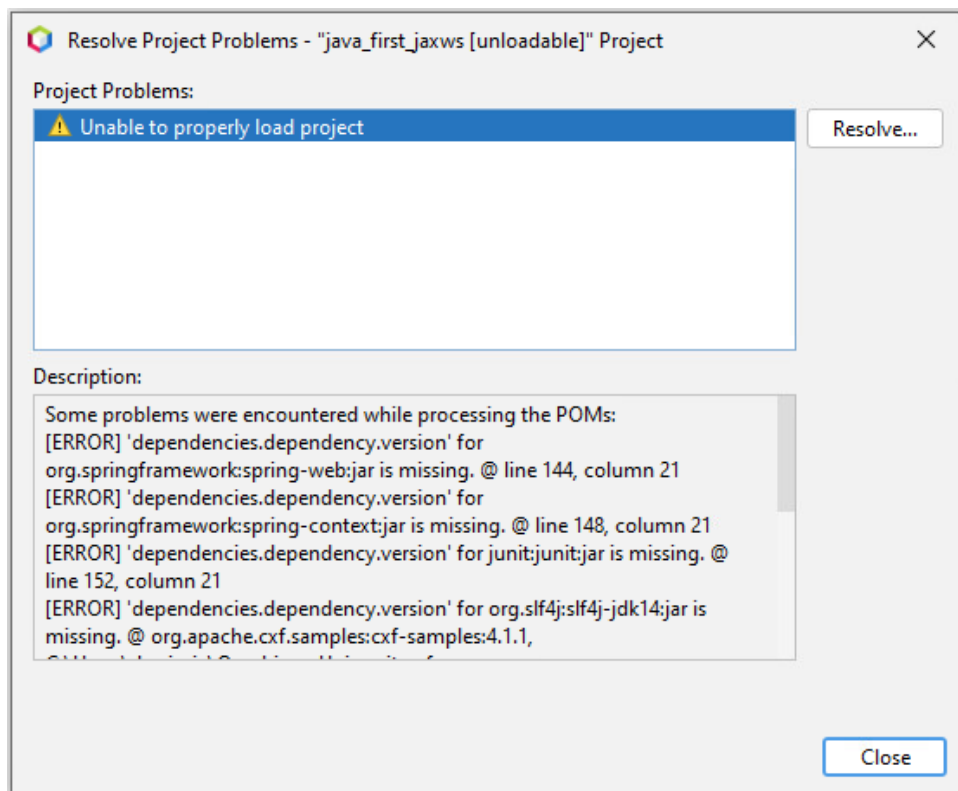
apache-cxf-4.1.1.zip - Apache CXF Example Projects (<https://cxf.apache.org/docs/sample-projects.html>).

This lab uses these two example projects:

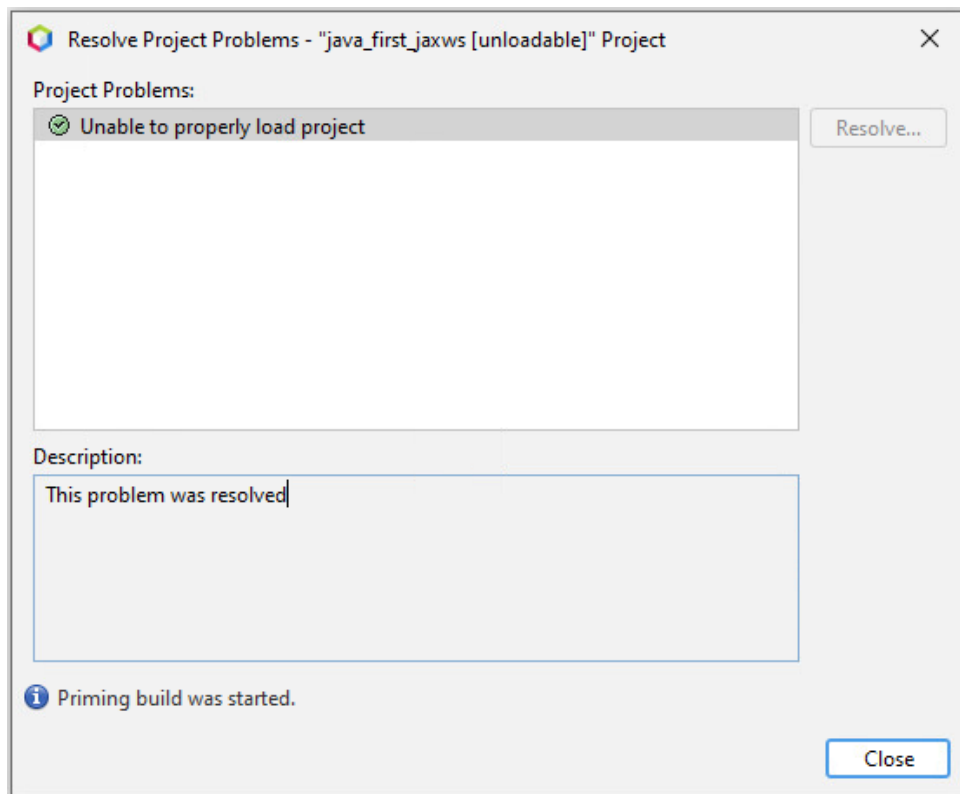
- **Java First using JAX-WS APIs.** This example illustrates how to develop a service using the "code first" approach with the JAX-WS APIs – this is for SOAP (Simple Object Access Protocol) web services. Project location: *apache-cxf-4.1.1\samples\java_first_jaxws*
- **JAX-RS Basic Demo.** This example shows a basic REST based web service using JAX-RS. Project location: *apache-cxf-4.1.1\samples\jax_rs\basic*

Activity 1 – Run SOAP web service example

1. Import the project **java_first_jaxws** into NetBeans. If it appears as *[unloadable]* (because of missing dependencies), right-click the project and select “Resolve Project Problems”:



Press “Resolve” – the problem should now be resolved:

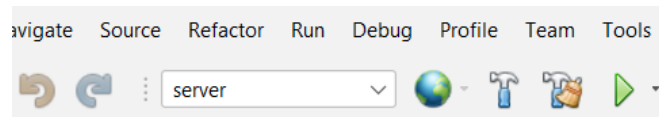


2. Explore the server and client code (**demo.hw.server** and **demo.hw.client**):

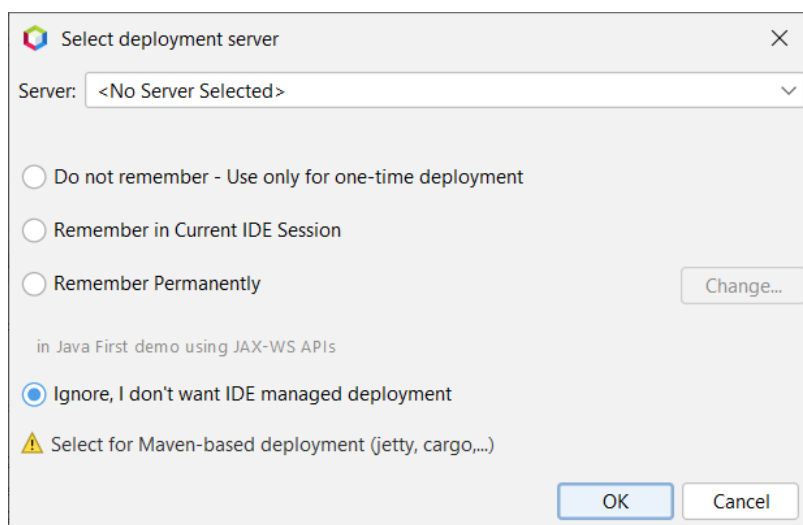
- The **HelloWorld** service maintains a map of users (**IntegerUserMap**)
- The client can add users and retrieve the map of users.

Run the server

3. Select the project configuration “server” and run the project:



4. When prompted to deploy to an application server, select “Ignore” (this allows the service to run standalone for the sake of simplicity):



5. The output should indicate the server is running on port 9000:

```
Starting Server
Mar 17, 2025 10:38:54 PM org.apache.cxf.wsdl.service.factory.ReflectionServiceFactoryBean buildServiceFromClass
INFO: Creating Service {http://server.hw.demo/}HelloWorld from class demo.hw.server.HelloWorld
Mar 17, 2025 10:38:54 PM org.apache.cxf.endpoint.ServerImpl initDestination
INFO: Setting the server's publish address to be http://localhost:9000/helloWorld
Mar 17, 2025 10:38:55 PM org.eclipse.jetty.server.Server doStart
INFO: jetty-12.0.16; built: 2024-12-09T21:02:54.535Z; git: c3f88bafb4e393f23204dc14dc57b042e84debc7; jvm 20.0.1+9-29
Mar 17, 2025 10:38:55 PM org.eclipse.jetty.server.AbstractConnector doStart
INFO: Started ServerConnector@6aa0779e{HTTP/1.1, (http/1.1)}{localhost:9000}
Mar 17, 2025 10:38:55 PM org.eclipse.jetty.server.Server doStart
INFO: Started oejs.Server@409ac55a{STARTING}[12.0.16,sto=0] @4110ms
Mar 17, 2025 10:38:55 PM org.eclipse.jetty.server.handler.ContextHandler doStart
INFO: Started oeje10s.ServletContextHandler@4a232261{ROOT,,b=null,a=AVAILABLE,h=oeje10s.ServletHandler@149e4583{STARTED}}
Mar 17, 2025 10:38:55 PM org.eclipse.jetty.ee10.servlet.ServletContextHandler doStart
INFO: Started oeje10s.ServletContextHandler@4a232261{ROOT,,b=null,a=AVAILABLE,h=oeje10s.ServletHandler@149e4583{STARTED}}
Server ready...
```

Note: The server automatically terminates after 5 minutes.

Run the client

6. Select the project configuration “client” and run the project (again, ignore the application server).
You should see output confirming the addition of users World, Galaxy and Universe:

```
Mar 17, 2025 10:54:27 PM org.apache.cxf.wsdl.service.factory.ReflectionServiceFactoryBean buildServiceFromClass
INFO: Creating Service {http://server.hw.demo/}HelloWorld from class demo.hw.server.HelloWorld
Mar 17, 2025 10:54:27 PM org.apache.cxf.wsdl.service.factory.ReflectionServiceFactoryBean buildServiceFromClass
INFO: Creating Service {http://server.hw.demo/}HelloWorld from class demo.hw.server.HelloWorld
Hello World
Hello World
Hello Galaxy
Hello Universe

Users:
1: World
2: Galaxy
3: Universe
```

7. Running the client again will add another copy of these users:

```
Mar 17, 2025 11:07:49 PM org.apache.cxf.wsdl.service.factory.ReflectionServiceFactoryBean buildServiceFromClass
INFO: Creating Service {http://server.hw.demo/}HelloWorld from class demo.hw.server.HelloWorld
Mar 17, 2025 11:07:49 PM org.apache.cxf.wsdl.service.factory.ReflectionServiceFactoryBean buildServiceFromClass
INFO: Creating Service {http://server.hw.demo/}HelloWorld from class demo.hw.server.HelloWorld
Hello World
Hello World
Hello Galaxy
Hello Universe

Users:
1: World
2: Galaxy
3: Universe
4: World
5: Galaxy
6: Universe
```

Tests

8. Review the test cases in **HelloWorldImplTest**, which run automatically when executing either the server or client:

```

-----
T E S T S
-----
Running demo.hw.server.HelloWorldImplTest
sayHiToUser called
sayHiToUser called
getUsers called
sayHi called
sayHiToUser called
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.063 s -- in demo.hw.server.HelloWorldImplTest

Results:

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

```

Activity 2 – Run RESTful web service example

1. Import the project **JAX-RS Basic Demo** into NetBeans. If necessary, resolve missing dependencies as in Activity 1.
2. Inspect the server and client code (**demo.jaxrs.server** and **demo.jaxrs.cilent**).

The example demonstrates a JAX-RS endpoint supporting GET, POST, PUT, and DELETE requests.

Example requests

A HTTP GET request to <http://localhost:9000/customerservice/customers/123> returns the customer with ID 123. XML response:

```

<Customer>
  <id>123</id>
  <name>John</name>
</Customer>

```

A HTTP GET request to <http://localhost:9000/customerservice/orders/223/products/323> returns product 323 that belongs to order 223. XML response:

```

<Product>
  <description>product 323</description>
  <id>323</id>
</Product>

```

A HTTP POST request to <http://localhost:9000/customerservice/customers> with the data:

```

<Customer>
  <name>Jack</name>
</Customer>

```

adds a customer named Jack.

A HTTP PUT request to URL <http://localhost:9000/customerservice/customers> with the data:

```

<Customer>
  <id>123</id>
  <name>John</name>

```

</Customer>

updates the customer with ID 123.

Run the server

3. Right-click **Server.java** -> "Run file". You should see output confirming the service is running on port 9000.

Important: Make sure that the SOAP server from Activity 1 is not still running on this port, otherwise you would get an error:

```
Mar 18, 2025 10:28:40 AM org.apache.cxf.endpoint.ServerImpl initDestination
INFO: Setting the server's publish address to be http://localhost:9000/
Mar 18, 2025 10:28:40 AM org.eclipse.jetty.server.Server doStart
INFO: jetty-12.0.16; built: 2024-12-09T21:02:54.535Z; git: c3f88bafb4e393f23204dc14dc57b042e84debc7; jvm 20.0.1+9-29
Mar 18, 2025 10:28:40 AM org.eclipse.jetty.server.AbstractConnector doStart
INFO: Started ServerConnector@710c2b53{HTTP/1.1, (http/1.1)}{localhost:9000}
Mar 18, 2025 10:28:40 AM org.eclipse.jetty.server.Server doStart
INFO: Started oejs.Server@7205765b{STARTING}[12.0.16,sto=0] @1023ms
Mar 18, 2025 10:28:40 AM org.eclipse.jetty.server.handler.ContextHandler doStart
INFO: Started oeje10s.ServletContextHandler@730d2164{ROOT,/,b=null,a=AVAILABLE,h=oeje10s.ServletHandler@24959ca4{STARTED}}
Mar 18, 2025 10:28:40 AM org.eclipse.jetty.ee10.servlet.ServletContextHandler doStart
INFO: Started oeje10s.ServletContextHandler@730d2164{ROOT,/,b=null,a=AVAILABLE,h=oeje10s.ServletHandler@24959ca4{STARTED}}
Server ready...
```

Run the client

4. Right-click **Client.java** -> "Run file". The console will display responses returned by the server:

```
Sent HTTP GET request to query customer info
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Customer><id>123</id><name>John</name></Customer>

Sent HTTP GET request to query sub resource product info
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Product><description>product 323</description><id>323</id></Product>

Sent HTTP PUT request to update customer info
Response status code: 200
Response body:

Sent HTTP POST request to add customer
Response status code: 200
Response body:
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Customer><id>124</id><name>Jack</name></Customer>
```

Activity 3 – Deploy web service

Deploy the service of Activity 1 to an application server. Detailed instructions are available here:

https://github.com/apache/cxf/tree/main/distribution/src/main/release/samples/java_first_jaxws

Activity 4 – Use REST API

A REST API that is useful for learning or testing is available at <https://restful-api.dev/>. Try the example requests shown on the website and implement a simple Java client that interacts with this API.

Useful resources

- <https://cxf.apache.org/>
- <https://restfulapi.net/>
- <https://restful-api.dev/>
- Public REST APIs: <https://www.postman.com/cs-demo/public-rest-apis/collection/tfzpqfc/public-rest-apis>
- Public SOAP APIs: <https://www.postman.com/cs-demo/public-soap-apis/collection/eebj1yq/public-soap-apis>